Universidad Veracruzana

Facultad de Instrumentación Electrónica

Ingeniería Biomédica

EE: Interfaces Hombre-Máquina

ACTIVIDAD 06. ACTUADORES EN EL PROCESO DE SALIDA DE INFORMACION DE UNA INTERFAZ HOMBRE-MAQUINA

Profesor:
Varela Lara Luis Julian

Integrantes del equipo *"ES3.14"*:
Alfonso Gamboa Rubén
Flores Montero Edsel Yetlanezi

Fecha: 5 de Junio de 2026

Xalapa, Enríquez, Ver.

I. Introducción al tema.

Las Interfaces Hombre-Máquina (HMI) han cambiado continuamente para satisfacer las necesidades de todo tipo de usuarios y han evolucionado más allá de la retroalimentación visual y los estímulos auditivos convencionales, abriendo paso a los sistemas de retroalimentación háptica como herramientas. Estos sistemas permiten recibir información espacial a través del sentido del tacto, convirtiéndose en una alternativa destinada a usuarios con limitaciones sensoriales o visuales. Al sustituir las señales ópticas por estímulos mecánicos localizados en la piel, es posible guiar las acciones motrices de un operador de manera intuitiva y en tiempo real. El núcleo del proyecto radica en recibir datos de desviación derecha-izquierda con respecto a una trayectoria y su transformación a una señal de intensidad vibratoria mediante actuadores independientes colocados en extremos corporales opuestos (izquierda y derecha). El operador, sin apoyo visual, percibe la dirección y la intensidad del de la desviación en forma de estímulos vibratorios, respondiendo inmediatamente a través de un joystick que regula la velocidad de corrección en tres niveles.

II. Marco teórico.

II-A. Interfaces Hápticas.

La tecnología háptica se basa en la estimulación de los mecanorreceptores cutáneos (como los corpúsculos de Pacini y Meissner) para transmitir datos. Para que una interfaz de este tipo funcione correctamente, el usuario debe recibir señales fácilmente diferenciables entre sí entre dos puntos del tacto y ofrecer variaciones detectables del estímulo, permitiendo que el sistema nervioso del usuario procese la señal y ejecute una respuesta psicomotora correctiva.

II-B. ESP32 vs Arduino.

Anteriormente, se había optado por utilizar el sistema Arduino Uno sin embargo para esta actividad final se propuso actual con el sistema ESP32 el cual afecta el comportamiento de la interfaz, de cuatro formas diferentes. Capacidad de Procesamiento y Concurrencia: Mientras que un Arduino clásico opera con un microcontrolador a 16 MHz en un solo núcleo, el ESP32 cuenta con una arquitectura de 32 bits a 240 MHz de doble núcleo. En sistemas interactivos de tiempo real que implementan un paradigma no bloqueante mediante la evaluación de milisegundos (millis()), la mayor cantidad de ciclos de reloj permite muestrear los dispositivos de entrada, procesar las tramas serie y modular las salidas táctiles con una latencia muy baja, evitando cuellos de botella.

II-B1. Resolución del Convertidor Analógico-Digital (ADC): El Arduino Uno posee un ADC de 10 bits que traduce el voltaje en un rango discreto de 0 a 1023. El ESP32 eleva esta resolución a 12 bits, generando una escala analógica mucho más amplia, de 0 a 4095. Esta mayor granularidad matemática altera los algoritmos de control, ya que permite definir con extrema precisión la ”zona muertaçentral del dispositivo de entrada y mapear con mayor finura los umbrales correspondientes a las velocidades requeridas.

II-B2. Modulación por Ancho de Pulso (PWM) Avanzada: A diferencia de la función analogWrite() de Arduino, el ESP32 maneja el PWM a través de un periférico especializado de control de LEDs (LEDC). Este módulo permite configurar por software tanto la frecuencia exacta de conmutación como la resolución en bits del ciclo de trabajo, lo que se traduce en una modulación de intensidad de vibración sumamente lineal, estable y libre de armónicos mecánicos indeseados.

II-B3. Niveles Lógicos y Restricciones de Corriente: Arduino opera de manera estándar a 5V, mientras que el ESP32 es un dispositivo de 3.3V. Esto exige que los potenciómetros de entrada se alimenten estrictamente a 3.3V para evitar el daño al microcontrolador. Adicionalmente, la corriente máxima que puede suministrar un pin GPIO del ESP32 está limitada a aproximadamente 12 mA.

II-C. Actuadores Hápticos.

Motores de Masa Rotatoria Excéntrica (ERM) Para la generación de la vibración se emplean micromotores de corriente continua del tipo Masa Rotativa Excéntrica. Estos dispositivos cuentan con un peso descentrado unido a su eje; al aplicar una diferencia de voltaje y corriente, la rotación de la masa descentrada genera una fuerza centrífuga asimétrica que se traduce en vibración mecánica. Una característica crítica de los motores ERM es que la frecuencia y la amplitud de la vibración están físicamente definidas: a mayor voltaje aplicado mediante la señal PWM, el motor gira más rápido (mayor frecuencia) y ejerce más fuerza (mayor amplitud).

II-D. Fenómenos de Transmisión Mecánica.

l comportamiento físico de las ondas mecánicas en materiales sólidos representa un desafío de diseño crítico en esté tipo de sistemas. Al intentar integrar el dispositivo de entrada y los actuadores vibratorios en un prototipo en impresión 3D fabricado en ácido poliláctico (PLA), se experimenta el fenómeno de cross-talk o acoplamiento háptico. El PLA, al ser un polímero termoplástico rígido y denso, actúa como un medio conductor mecánico de baja amortiguación, propagando las ondas elásticas generadas por cualquiera de los motores a lo largo de toda la estructura continua. Como consecuencia directa, los estímulos táctiles de los canales izquierdo y derecho se mezclan en la carcasa, provocando que el origen espacial de la señal resulte mecánicamente irreconocible para el usuario. A este inconveniente se suma que las tolerancias obstruyen el libre desplazamiento de la palanca, limitando su recorrido angular e impidiendo alcanzar las lecturas de voltaje necesarias para activar las velocidades analógicas más altas en el código. Para mitigar estos efectos, se retiraron los elementos qué causan la interferencia, prescindiendo de la carcasa unificada y aislando los motores de vibración. Al sostener un motor libre en cada mano, se aprovecha la separación anatómica del cuerpo humano como un filtro de señales, restaurando la discriminación estéreo de la información háptica y liberando al joystick de cualquier obstrucción física.

III. Desarrollo de la actividad.

III-A. Librerías.

Como anteriormente se mencionó, se trabajó en el framework *platformIO* en un *ESP32 (classic)* en lugar de arduino con Arduino IDE, por lo que se tiene que imporar la librería *<Arduino.h>*.

III-B. Variables y constantes

III-B1. Configuración de pines para ESP32: Dentro de esta sección declaramos los pines que vamos a usar apoyándonos de la documentación de los pines del ESP32 ([1]):

- **GPIO34.** Este pin es únicamente entrada analógica [1], por eso podemos usarla para leer lo que nos entrega VRy del Joystick (**Nota:** ocupamos el eje Y del Joystick debido a que resultó ser más cómodo,lo que llevó a reescribir el código).
- **GPIO18 y GPIO19.** Estos pines al poder soportar las PWM, se utilizaron para modular la vibración de nuestros motores-vibradores.

III-C. Control del tiempo

Como aprendimos en anteriores prácticas, podemos usar *unsigned long* para poder guardar el tiempo actual, el tiempo anterior y el periodo en el queremos que se repita nuestra acción, pudiendo manejar numeros muy grandes y aprovechando todos los espacios de bits gracias a que no estamos ocupando números negativos.

III-C1. Constantes de velocidad: Esta sección nos permite definir la velocidad de impresión (periodo en milisegundos) desde el principio del código, lo que permite una configuración más sencilla; pues la configuración de velocidad fue manual, probando una por una para determinar cuál es más cómoda.

III-C2. Constantes para la intensidad de vibración: Esta sección tambien nos funciona como configuración pero a la velocidad a la que vibran nuestros motores-vibradores, pudiendo tener virbaciones altas y bajas.

III-C3. setup(): Primeramente se inicia la comunciación serial a 115200 baudios, pues es el estándar con el ESP32.

Establecemos una resolución de 12 bits, para una lectura analógica del pin del Joystick,

Configuramos los pines de los motores como pines *OUTPUT*

III-C4. loop(): Para el *loop* se utilizaron los conocimientos adquiridos de las anteriores prácticas, utilizando *CASE* y una máquina de estados para evitar el bloqueo, siendo que unicamente cambiamos el estado de nuestra máquina para manejar según el ángulo de nuestro joystick. Igual que en anteriores prácticas, eivtamos el bloqueo que causa *break* utilizando *milis()* en su lugar.

III-D. Código completo

A continuación se deja el código completo usado durante el desarrollo de esta actividad:

```
#include <Arduino.h>

// --- Configuración de Pines para ESP32 ---
const int VRy = 34;       // Pin analógico (ADC1_CH6) para el eje y del Joystick
const int motorIzq = 18;  // Pin GPIO para motor vibrador izquierdo (Soporta PWM)
const int motorDer = 19;  // Pin GPIO para motor vibrador derecho (Soporta PWM)

// --- Variables para control de tiempo ---
unsigned long tiempoActual;
unsigned long tiempoAnteriorY = 0;
unsigned long periodoY = 0;

// --- Constantes de velocidad (Periodo en milisegundos entre envíos) ---
const unsigned long VEL_BAJA = 300;
const unsigned long VEL_MEDIA = 120;
const unsigned long VEL_ALTA = 30;
const unsigned long VEL_TURBO = 8;

// --- Constantes para Intensidad de Vibración (PWM de 0 a 255) ---
const int VIB_BAJA = 100;
const int VIB_MEDIA = 175;
const int VIB_ALTA = 255;
const int VIB_NULA = 0;

void setup() {
  Serial.begin(115200);

  // Configuración del pin analógico en el ESP32 (Resolución a 12 bits: 0-4095)
  analogReadResolution(12);
  pinMode(VRy, INPUT);

  // Configuración de pines de los motores
  pinMode(motorIzq, OUTPUT);
  pinMode(motorDer, OUTPUT);

  // Asegurarnos de que inician apagados
  analogWrite(motorIzq, VIB_NULA);
  analogWrite(motorDer, VIB_NULA);
}

void loop() {
  tiempoActual = millis();

  // 1. LECTURA DE COMANDOS SERIE (Retroalimentación Háptica)
  if (Serial.available() > 0) {
    int comandoRecibido = Serial.read();

    switch (comandoRecibido) {
      // --- Desviación hacia la Izquierda ---
      case 50: // BAJA IZQUIERDA
        analogWrite(motorIzq, VIB_BAJA);
        analogWrite(motorDer, VIB_NULA);
        break;
      case 60: // MEDIA IZQUIERDA
        analogWrite(motorIzq, VIB_MEDIA);
        analogWrite(motorDer, VIB_NULA);
        break;
      case 70: // ALTA IZQUIERDA
        analogWrite(motorIzq, VIB_ALTA);
        analogWrite(motorDer, VIB_NULA);
        break;
```

```
    // --- Desviación hacia la Derecha ---
    case 80: // BAJA DERECHA
      analogWrite(motorIzq, VIB_NULA);
      analogWrite(motorDer, VIB_BAJA);
      break;
    case 90: // MEDIA DERECHA
      analogWrite(motorIzq, VIB_NULA);
      analogWrite(motorDer, VIB_MEDIA);
      break;
    case 100: // ALTA DERECHA
      analogWrite(motorIzq, VIB_NULA);
      analogWrite(motorDer, VIB_ALTA);
      break;

    // --- Sin Desviación ---
    case 200:
      analogWrite(motorIzq, VIB_NULA);
      analogWrite(motorDer, VIB_NULA);
      break;
  }
}

// 2. LECTURA DEL JOYSTICK Y ENVÍO DE COMANDOS (Control Vehículo)
int Y = analogRead(VRy);
int estadoY = 0;          // 0: Centro, 1: Derecha (10), -1: Izquierda (20)

// Evaluar hacia la DERECHA
if (Y > 2200) {
  estadoY = 1;
  if (Y < 2800) {
    periodoY = VEL_BAJA;
  } else if (Y < 3400) {
    periodoY = VEL_MEDIA;
  } else if (Y < 3900) {
    periodoY = VEL_ALTA;
  } else {
    periodoY = VEL_TURBO; // Palanca totalmente a la derecha
  }
}
// Evaluar hacia la IZQUIERDA
else if (Y < 1800) {
  estadoY = -1;
  if (Y > 1200) {
    periodoY = VEL_BAJA;
  } else if (Y > 600) {
    periodoY = VEL_MEDIA;
  } else if (Y > 150) {
    periodoY = VEL_ALTA;
  } else {
    periodoY = VEL_TURBO; // Palanca totalmente a la izquierda
  }
}

// Gestión del envío de comandos por el puerto serie
if (estadoY != 0) {
  if (tiempoActual - tiempoAnteriorY >= periodoX) {
    if (estadoY == 1) {
      Serial.write(10); // Comando para mover a la derecha
    } else if (estadoY == -1) {
      Serial.write(20); // Comando para mover a la izquierda
    }
    tiempoAnteriorY = tiempoActual;
  }
 }
}
```

III-E. Armado de Circuito

Las conexiones se realizaron estañando puntas dupont a las puntas de los motores vibradores (compartiendo el uno para GND), simplemente conectando estos pintes como se aprecia en la figura 1.

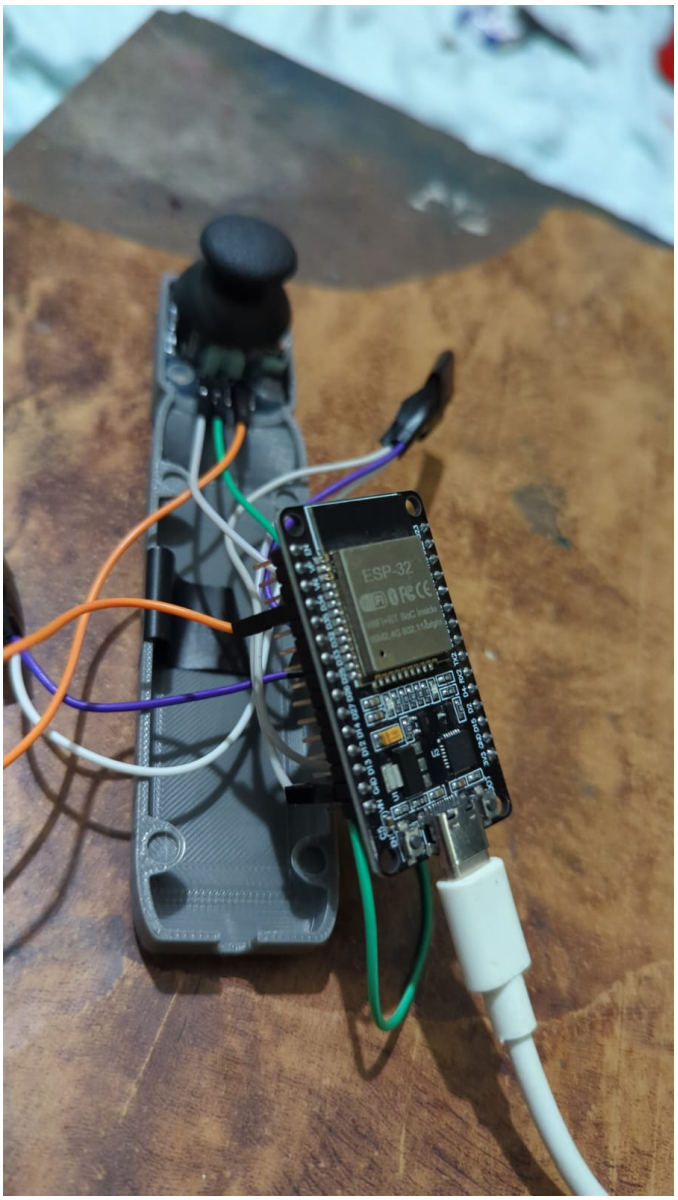


Figura 1. Vista general del circuito armado

III-F. Conexiones y pines

-

IV. Conclusiones

Para esta práctica fue bastante útil la experiencia previa ganada en otras actividades. Así mismo, el registro de solo dos direcciones simplificó el sistema de forma sustancial en comparación a otras actividades donde los vectores compuestos por dos direcciones requerían una definición y un comportamiento más preciso. Sin embargo, la decisión de intentar crear un prototipo para embeber esté sistema, a pesar de qué su ergonomía y comodidad de agarre fue positiva, el uso de un material inadecuado y un diseño carente de las consideraciones de los fenómenos físicos representó un reto adicional al tener qué modificar el mismo para lograr qué esté se volviera operable, lo cual denota la importancia de considerar todos elementos adecuados al momento de diseñar una interfaz Hombre-Máquina. Más allá de las complicaciones al agregar un elemento extra, esta actividad demuestra la enorme flexibilidad de opciones y combinaciones qué están disponibles para ofrecer una mejor experiencia a los usuarios, incluso cuando algunas funciones sensoriales pueden estar comprometidas.

V. firma de revisión

Debido a complicaciones climáticas se anexa el link del video con la evidencia de que el código cumple con la prueba “seguir ruta”; necesaria para validar la actividad: https://www.youtube.com/shorts/WbHFZQiSFzA

Referencias

[1] L. Llamas. «Pinout y detalles del hardware del ESP32,» visitado 5 de jun. de 2026. dirección: https://www.luisllamas.es/esp32-detalles-hardware-pinout/